Unification

Concepts of Programming Languages

CAS CS 320

Recap

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$
2. Use the $\mathscr{C}$ to determine the "actual" type of e in Γ

Recall: Constraint-Based Inference

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

The constraint-based type inference process:

1. Derive $\Gamma \vdash e : \tau$ *relative to some constraints* $\mathscr{C}$
2. Use the $\mathscr{C}$ to determine the "actual" type of e in Γ

today (and next week)

Recall: HM$^-$ (Typing)

$$\frac{n \texttt{ is an integer}}{\Gamma \vdash n : \mathsf{int} \dashv \varnothing} \ \texttt{(int)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \Gamma \vdash e_3 : \tau_3 \dashv \mathscr{C}_3}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau_3 \dashv \tau_1 \doteq \mathsf{bool}, \tau_2 \doteq \tau_3, \mathscr{C}_1, \mathscr{C}_2, \mathscr{C}_3} \ \texttt{(if)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 = e_2 : \mathsf{bool} \dashv \tau_1 \doteq \tau_2, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(eq)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 + e_2 : \mathsf{int} \dashv \tau_1 \doteq \mathsf{int}, \tau_2 \doteq \mathsf{int}, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(add)}$$

$$\frac{\alpha \texttt{ is fresh} \qquad \Gamma, x : \alpha \vdash e : \tau \dashv \mathscr{C}}{\Gamma \vdash \lambda x . e : \alpha \to \tau \dashv \mathscr{C}} \ \texttt{(fun)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \alpha \texttt{ is fresh}}{\Gamma \vdash e_1 e_2 : \alpha \dashv \tau_1 \doteq \tau_2 \to \alpha, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(app)}$$

Recall: What is a constraint?

$$\tau_1 \doteq \tau_2$$

A **type constraint** is a predicate on types. The only kind we will consider:

"τ_1 should be the same as τ_2"

Enforcing a constraint like this is called **unifying** τ_1 and τ_2

Unification

High Level

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to \mathsf{int} \to \mathsf{int} &\doteq b \to c
\end{aligned}
$$

Unification is the process of solving a system of equations over *symbolic* expressions

Type Unification

$\sigma ::= \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma$

A **type unification problem** is a collection of equations of the form

$$\begin{aligned} s_1 &\doteq t_1 \\ s_2 &\doteq t_2 \\ &\vdots \\ s_k &\doteq t_k \end{aligned}$$

where $s_1, \ldots, s_k$ and $t_1, \ldots, t_k$ are types

Unifiers

Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

$$\begin{aligned} \mathscr{S} t_1 &= \mathscr{S} s_1 \\ \mathscr{S} s_2 &= \mathscr{S} t_2 \\ &\vdots \\ \mathscr{S} s_k &= \mathscr{S} t_k \end{aligned}$$

Unifiers

A **solution** or **unifier** is a sequence of substitutions to *some* of the type parameters appearing in $\mathscr{U}$, typically written

$$\mathscr{S} = [x_1 \mapsto t_1, x_2 \mapsto t_2, \ldots, x_n \mapsto t_n]$$

that it **satisfies** every equation

$$\begin{aligned} \mathscr{S} t_1 &= \mathscr{S} s_1 \\ \mathscr{S} s_2 &= \mathscr{S} t_2 \\ &\vdots \\ \mathscr{S} s_k &= \mathscr{S} t_k \end{aligned}$$

where $\mathscr{S} t = [t_n/x_n] \ldots [t_1/x_1] t$

Most General Unifiers

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to \mathsf{int} \to \mathsf{int} &\doteq b \to c
\end{aligned}
$$

Most General Unifiers

A **most general unifier** is a solution $\mathcal{S}$ such that, for any solution $\mathcal{S}'$, there is another solution $\mathcal{S}''$ such that $\mathcal{S}'\tau = \mathcal{S}''\mathcal{S}\tau$

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to \mathsf{int} \to \mathsf{int} &\doteq b \to c
\end{aligned}
$$

Most General Unifiers

A **most general unifier** is a solution $\mathcal{S}$ such that, for any solution $\mathcal{S}'$, there is another solution $\mathcal{S}''$ such that $\mathcal{S}'\tau = \mathcal{S}''\mathcal{S}\tau$

In other words, $\mathcal{S}'$ is $\mathcal{S}$ *with more substitutions*

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to (\mathsf{int} \to \mathsf{int}) &\doteq b \to c
\end{aligned}
$$

$$
\mathcal{S}' = \begin{cases} b \mapsto \mathsf{int} \\ c \mapsto \mathsf{int} \to \mathsf{int} \\ d \mapsto \mathsf{int} \\ a \mapsto \mathsf{int} \to \mathsf{bool} \\ e \mapsto \mathsf{bool} \end{cases}
$$

int → bool = int → bool

int → int = int → int

int → (int → int) = int → (int → int) ✓

Not a MGU

Most General Unifiers

A **most general unifier** is a solution $\mathcal{S}$ such that, for any solution $\mathcal{S}'$, there is another solution $\mathcal{S}''$ such that $\mathcal{S}'\tau = \mathcal{S}''\mathcal{S}\tau$

In other words, $\mathcal{S}'$ is $\mathcal{S}$ *with more substitutions*

$$
\begin{aligned}
a &\doteq d \to e \\
c &\doteq \mathsf{int} \to d \\
\mathsf{int} \to (\mathsf{int} \to \mathsf{int}) &\doteq b \to c
\end{aligned}
$$

$$
S' = \begin{cases} b \mapsto \mathsf{int} \\ c \mapsto \mathsf{int} \to \mathsf{int} \\ d \mapsto \mathsf{int} \\ a \mapsto \mathsf{int} \to e \end{cases}
$$

$\mathsf{int} \to e = \mathsf{int} \to e$

$\mathsf{int} \to \mathsf{int} = \mathsf{int} \to \mathsf{int}$

$\mathsf{int} \to (\mathsf{int} \to \mathsf{int}) = \mathsf{int} \to (\mathsf{int} \to \mathsf{int})$ ✓

Is a MGU

An Algorithm (High Level)

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

- (S) syntactical equality (e.g., int ≐ int)

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

- (S) syntactical equality (e.g., int $\doteq$ int)
- (F) function type equality (e.g., $\alpha \to \beta \doteq \alpha \to \gamma$)

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

- (S) syntactical equality (e.g., int $\doteq$ int)
- (F) function type equality (e.g., $\alpha \to \beta \doteq \alpha \to \gamma$)
- (A) assignment (e.g., $\alpha \doteq \mathsf{int} \to \beta$)

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

- (S) syntactical equality (e.g., int $\doteq$ int)
- (F) function type equality (e.g., $\alpha \to \beta \doteq \alpha \to \gamma$)
- (A) assignment (e.g., $\alpha \doteq \mathsf{int} \to \beta$)

Assignment become *part of our solution*

An Algorithm (High Level)

Process equations one at a time, updating *the collection of equations*

We **FAIL** if we reach an unsatisfiable equation

The three kinds of *satisfiable* equations:

- (S) syntactical equality (e.g., int $\doteq$ int)
- (F) function type equality (e.g., $\alpha \to \beta \doteq \alpha \to \gamma$)
- (A) assignment (e.g., $\alpha \doteq \text{int} \to \beta$)

Assignment become *part of our solution*

We're guaranteed to get the a most general unifier

An Algorithm (Pseudocode)

An Algorithm (Pseudocode)

in: type unification problem $\mathscr{U}$
out: most general unifier for $\mathscr{U}$

An Algorithm (Pseudocode)

in: type unification problem $\mathscr{U}$
out: most general unifier for $\mathscr{U}$

$\mathscr{S} \leftarrow$ empty solution

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty
```

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty

  MATCH eq:
```

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

set difference : "remove eq from $\mathcal{U}$"

MATCH eq:

(S) $t \doteq t \implies \mathcal{U} \leftarrow \boxed{\mathcal{U} \setminus \{eq\}}$ // if *syntactically* equal then remove eq

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

An Algorithm (Pseudocode)

```
in: type unification problem 𝒰
out: most general unifier for 𝒰

𝒮 ← empty solution

WHILE eq ∈ 𝒰:  // 𝒰 is not empty

  MATCH eq:

    (S) t ≐ t ⟹ 𝒰 ← 𝒰\{eq}  // if syntactically equal then remove eq

    (F) s₁ → t₁ ≐ s₂ → t₂ ⟹ 𝒰 ← 𝒰\{eq} ∪ {s₁ ≐ s₂, t₁ ≐ t₂} // remove eq and add s₁ ≐ s₂ and t₁ ≐ t₂

    (A) α ≐ t or t ≐ α where α ∉ FV(t) ⟹ // type variable α does not appear free in t
```

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

    OTHERWISE $\implies$ **FAIL**

An Algorithm (Pseudocode)

in: type unification problem $\mathcal{U}$
out: most general unifier for $\mathcal{U}$

$\mathcal{S} \leftarrow$ empty solution

WHILE $eq \in \mathcal{U}$: // $\mathcal{U}$ is not empty

  MATCH eq:

    (S) $t \doteq t \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$ // if *syntactically* equal then remove eq

    (F) $s_1 \to t_1 \doteq s_2 \to t_2 \implies \mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\} \cup \{s_1 \doteq s_2, t_1 \doteq t_2\}$ // remove eq and add $s_1 \doteq s_2$ and $t_1 \doteq t_2$

    (A) $\alpha \doteq t$ or $t \doteq \alpha$ where $\alpha \notin \mathsf{FV}(t) \implies$ // type variable α does not appear free in t

      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\alpha \mapsto t\}$ // add $\alpha \mapsto t$ to $\mathcal{S}$

      doesn't appear on RHS

      $\mathcal{U} \leftarrow \mathcal{U} \backslash \{eq\}$

      perform the substitution $\alpha \mapsto t$ to every equation in $\mathcal{U}$

    OTHERWISE $\implies$ **FAIL**

RETURN $\mathcal{S}$

Example

$S = [$

$a \mapsto d \to e$

$c \mapsto \text{int} \to d$

$b \mapsto \text{int}$

$d \mapsto \text{int}$

$]$

$\cancel{a \doteq d \to e}$ (A)

$\cancel{c \doteq \text{int} \to d}$ (A)

$\cancel{\text{int} \to \text{int} \to \text{int} \doteq b \to \cancel{c}}$ (F)

$(\text{int} \to d)$

$\cancel{\text{int} \doteq b}$ (A)

$\cancel{\text{int} \to \text{int} \doteq \text{int} \to d}$ (F)

$\cancel{\text{int} \doteq \text{int}}$ (S)

$\cancel{\text{int} \doteq d}$ (A

Example

$$a \doteq d \to e$$
$$c \doteq \text{int} \to d$$
$$\text{int} \to \text{int} \to \text{int} \doteq b \to c$$

$S = [$

$a \mapsto d \to e$

$c \mapsto \text{int} \to d$

$b \mapsto \text{int}$

$d \mapsto \text{int}$

$]$

$Sa =$

$[\text{int}/d][\text{int}/b][\text{int} \to d/c][d \to e/a]a$

$$a \xrightarrow{a \mapsto d \to e} d \to e \xrightarrow{c \mapsto \text{int} \to d} d \to e \xrightarrow{b \mapsto \text{int}} d \to e \xrightarrow{d \mapsto \text{int}} \text{int} \to e$$

$$d \to e \;\; \ldots \xrightarrow{d \mapsto \text{int}} \text{int} \to e$$

$$Sa = S(d \to e)$$

fun x → x x

Unification may Fail

$\cancel{a \doteq b \to c}$ (A)

$b \doteq \cancel{a} \to \text{int}$ (A)

(b → c)

FAIL

$b \in FV((b \to c) \to \text{int})$

$\cancel{S = \{ a \mapsto b \to c}$

workshop